

Reinforcement learning with human feedback (RLHF): The "chat" in chatbot

LIN 313 Language and Computers
Fall 2025

Admin

- Homework 4 out: Due next Thursday, 11/20
- Readings this week in Mitchell

What do we need to understand ChatGPT?

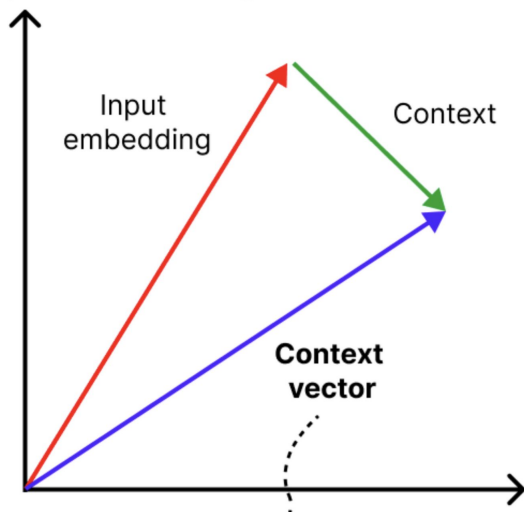
- language modeling task
- probability
- perceptrons
- neural networks
- deep learning (Multi-layer perceptrons)
- attention
- contextual embeddings
- pre-training
- decoder-only model (GPTs, all LLMs)
- context-window
- RLHF (reinforcement learning with human feedback)

What do we need to understand ChatGPT?

- language modeling task
- probability
- perceptrons
- neural networks
- deep learning (Multi-layer perceptrons)
- attention
- contextual embeddings
- pre-training
- decoder-only model (GPTs, all LLMs)
- context-window
- RLHF (reinforcement learning with human feedback)

Contextual Word Embeddings

Attention mechanism: Convert all input embeddings to context vector

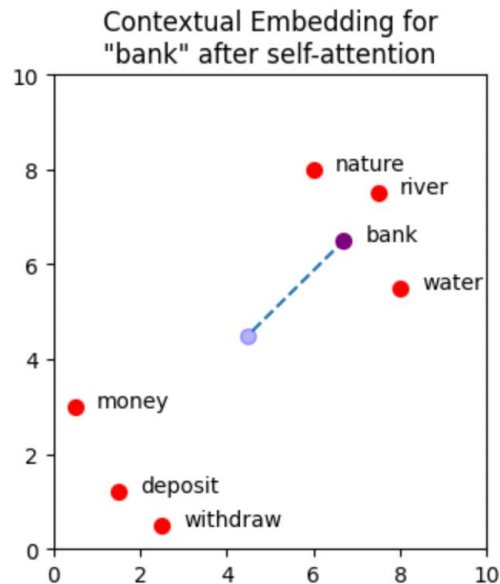
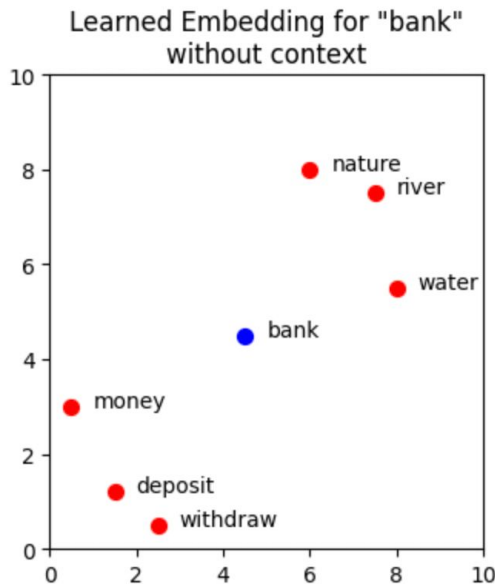


This is much richer. Because it has information about neighboring tokens.

Contextual Word Embeddings

Where we used to have one embedding vector per word *type*, we now have one embedding per word *token*.

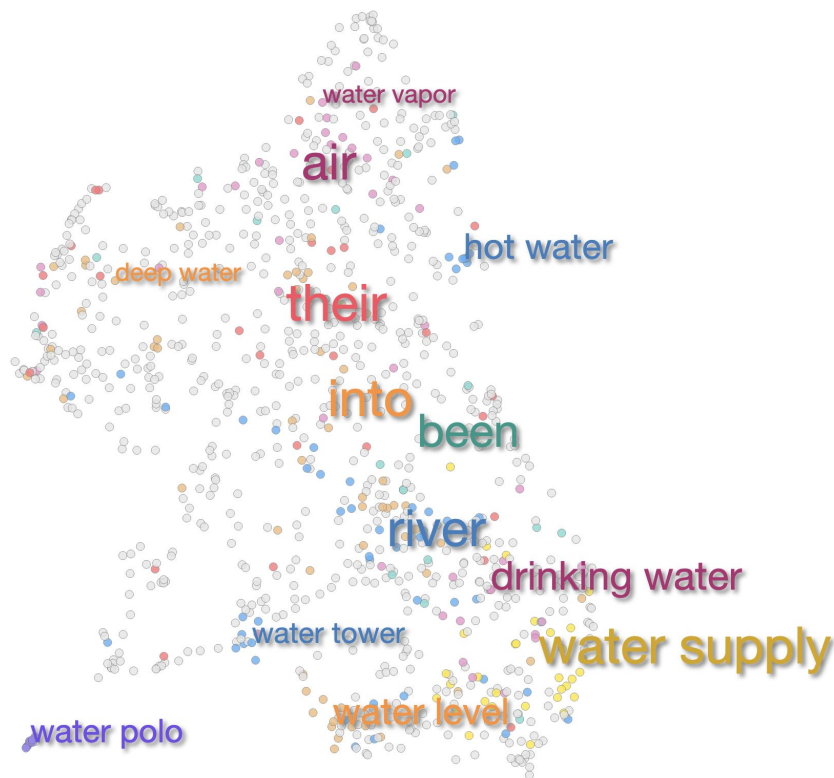
You can think of attention as 'looking up' which meaning of the word we are dealing with, or 'soaking up' information about the context



Contextual Word Embeddings

Where we used to have one embedding vector per word *type*, we now have one embedding per word *token*.

You can think of attention as 'looking up' which meaning of the word we are dealing with, or 'soaking up' information about the context



demo:

<https://storage.googleapis.com/bert-wsd-vis/demo/index.html?#word=water>

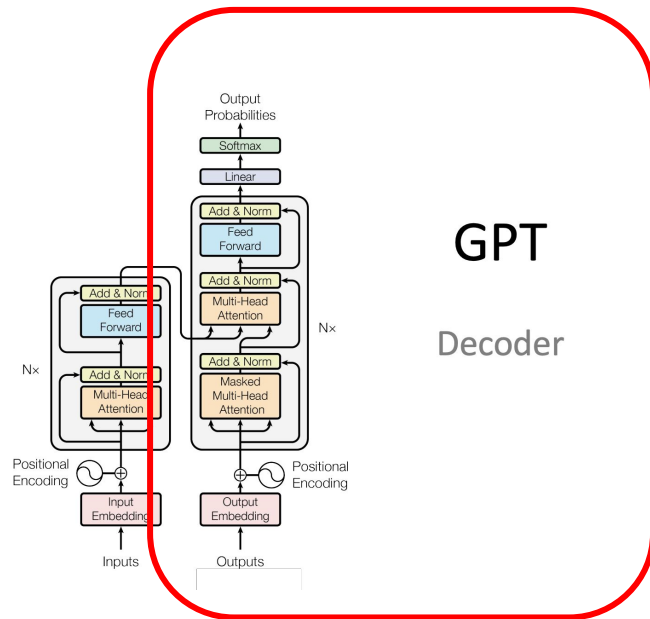
Improving Language Understanding by Generative Pre-Training (Sutskever et al., 2018) (OpenAI)

AKA GPT-1 !

- Task: next word prediction
- **decoder-only** architecture (the part of the encoder-decoder that generates new words)
- 117 million parameters
- 768-dimensional embeddings
- fixed context window : look back at the last 2048 tokens

BERT

Encoder



GPT

Decoder

GPT Didn't get that much attention at first

- It's designed to *generate* text, but what researchers really cared about is *classifying* text
 - GPT-1 was not a good text generator. It lost the thread of the conversation very quickly
 - It couldn't be fine-tuned for classification the way that encoder-only models were.

The solution? Make it bigger

GPT Didn't get that much attention at first

- It's designed to *generate* text, but what researchers really cared about is *classifying* text
 - GPT-1 was not a good text generator. It lost the thread of the conversation very quickly
 - It couldn't be fine-tuned for classification the way that encoder-only models were.

The solution? Make it bigger

Model	Parameters	Layers	Attention Heads	Embedding Dim	Training Data
GPT-1	117M	12	12	768	BooksCorpus
GPT-2	1.5B	48	25	1,600	WebText
GPT-3	175B	96	96	12,288	Mixed Corpus
GPT-4	~1.7T*	120+	128+	20,480+	Proprietary

*Estimated. GPT-4's architecture remains undisclosed but emphasizes sparse attention for efficiency.

GPT-3

Meet GPT-3. It Has Learned to Code (and Blog and Argue).

The latest natural-language system generates tweets, pens poetry, summarizes emails, answers trivia questions, translates languages and even writes its own computer programs.



- text completions
- no "fine-tuning": simply predicts the next word

RLHF: The "chat" in ChatGPT

The Problem: Base LLMs can predict text, but that's not enough

- A model trained on internet text learns to complete patterns.
- We need to tweak it so that it **responds** to the user rather than finishing their sentences / guessing what the user will say next.
- Also: it might generate undesired/toxic content, hallucinate, or give unhelpful responses

The idea: Train the LLM on any and all text. Then, **fine-tune** the model by training on ideal conversations written by people.

- This is waaay to hard and expensive. Enter RLHF

Reinforcement Learning (RL)

RLHF: an **agent**—the learning program—performs actions in an **environment** (usually a computer simulation) and occasionally receives **rewards** from the environment.

- Reinforcement learning is different from the kinds of machine learning we have talked about so far because it **requires no labeled training examples**.
- common paradigm: have robots or algorithms play hundreds or thousands of hours of simulated games.
 - Deep

Operant conditioning

a learning process where **behavior is shaped by its consequences**—essentially, we learn to do things that lead to rewards and avoid things that lead to punishment.

- **core principle:** behaviors followed by positive consequences become more frequent, while behaviors followed by negative consequences become less frequent.

B.F. Skinner developed operant conditioning in the 1930s-1950s using his famous "Skinner box"—where rats or pigeons learned to press levers for food rewards.

Operant Conditioning: 4 Types

1. Positive Reinforcement: Add something desirable → behavior increases

Example: Study hard → get praised → study more

2. Negative Reinforcement: Remove something unpleasant → behavior increases

Example: Take aspirin → headache goes away → take aspirin again next time

(Note: This is NOT punishment—it increases behavior!)

3. Positive Punishment: Add something unpleasant → behavior decreases

Example: Touch hot stove → feel pain → stop touching hot stoves

4. Negative Punishment: Remove something desirable → behavior decreases

Example: Break curfew → lose phone privileges → follow curfew

Sony Aibo

2005 Robocup
Final



Meet Rosie, the robot dog.

With reinforcement learning, the robo-dog can learn flexible strategies on its own.

Rosie is very simple. She knows her **state**, and can perform a few **actions**.

state: how many steps from the ball am I?

actions: kick, forward, back, wag

Other than that, she knows nothing.



Training Rosie

We will use positive reinforcement only, rewarding Rosie when she kicks the ball.

We will train Rosie in *episodes*.

Each of Rosie's actions is one *iteration*.

Each training episode will continue until Rosie gets a *reward*.

Episode 1

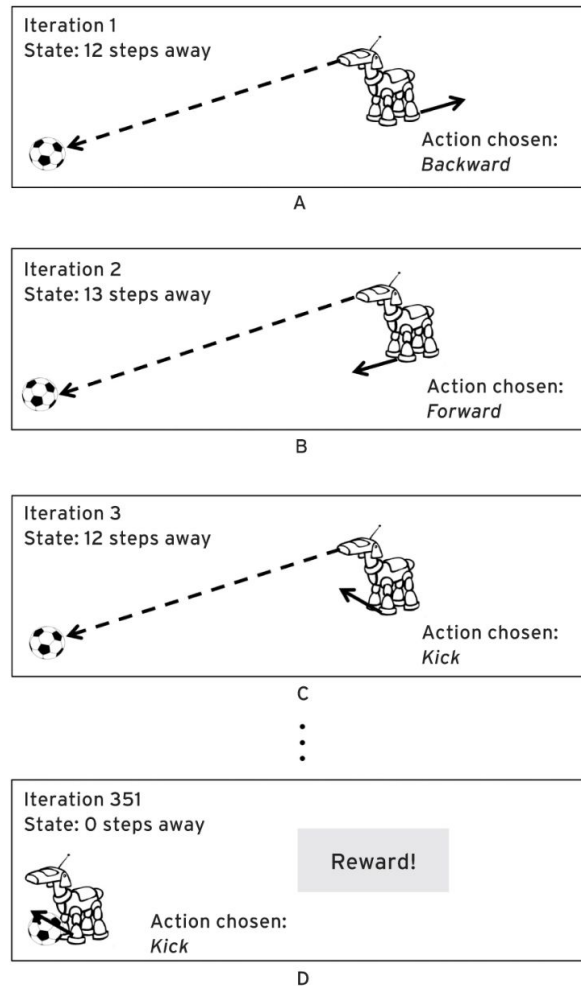


FIGURE 23: A hypothetical first episode of reinforcement learning

Episode 1

Encoding Desire: the Q-table

Q-table:					
State	0 steps away	1 step away	...	10 steps away	...
Action	Forward 0	Forward 0	...	Forward 0	...
	Backward 0	Backward 0		Backward 0	
	Kick 10	Kick 0		Kick 0	

FIGURE 24: Rosie's Q-table after her first episode of reinforcement learning

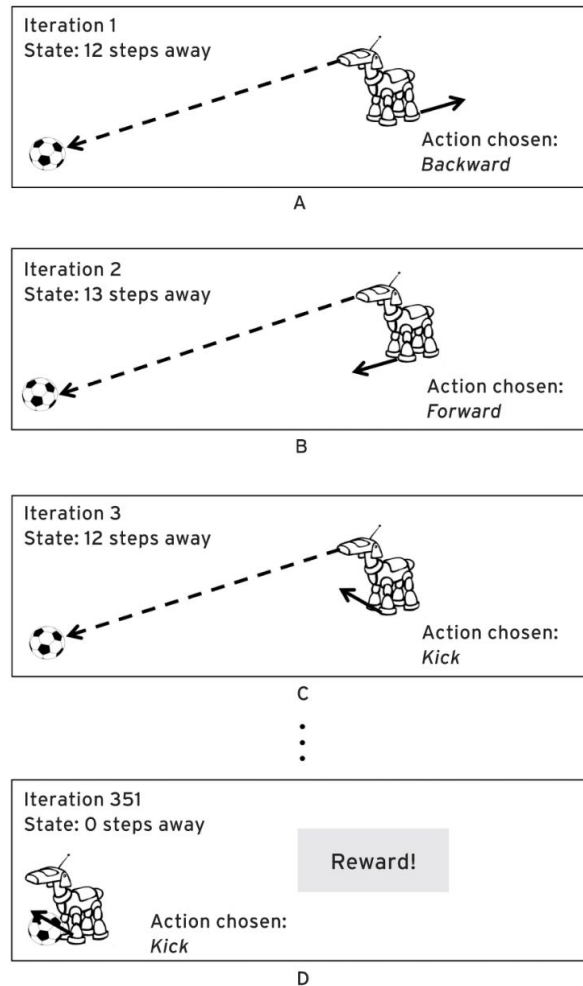


FIGURE 23: A hypothetical first episode of reinforcement learning

Episode 2

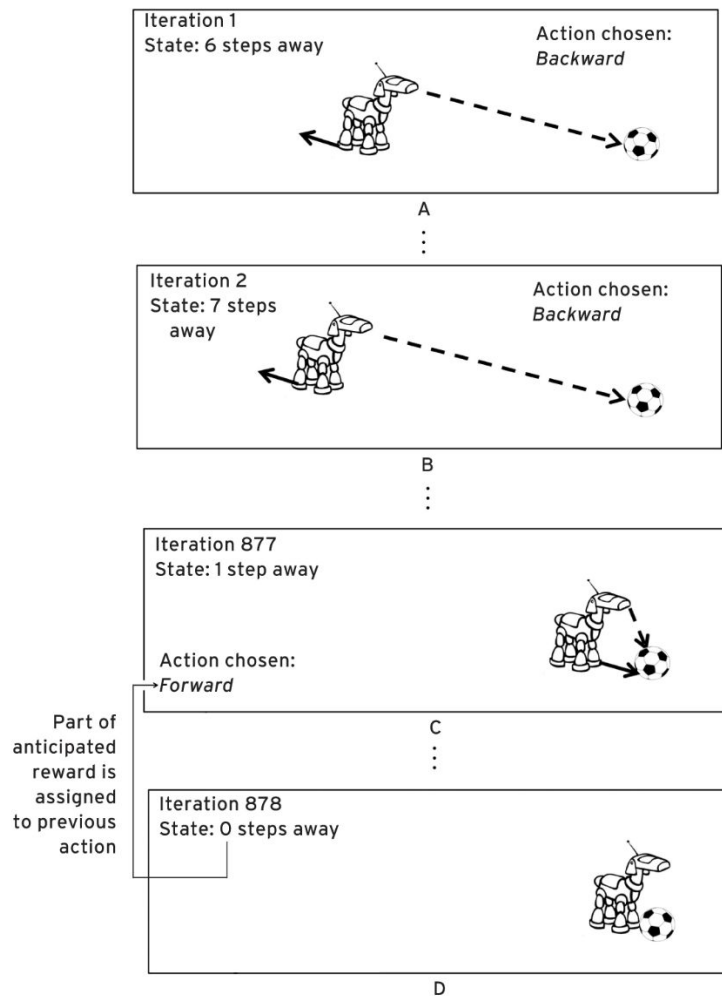


FIGURE 25: The second episode of reinforcement learning

Episode 2

Go to page 139

Q-table:					
State	0 steps away		1 step away	...	10 steps away ...
Action	Forward	0	Forward	8	...
	Backward	0	Backward	0	...
	Kick	10	Kick	0	...

FIGURE 26: Rosie's Q-table after her second episode of reinforcement learning

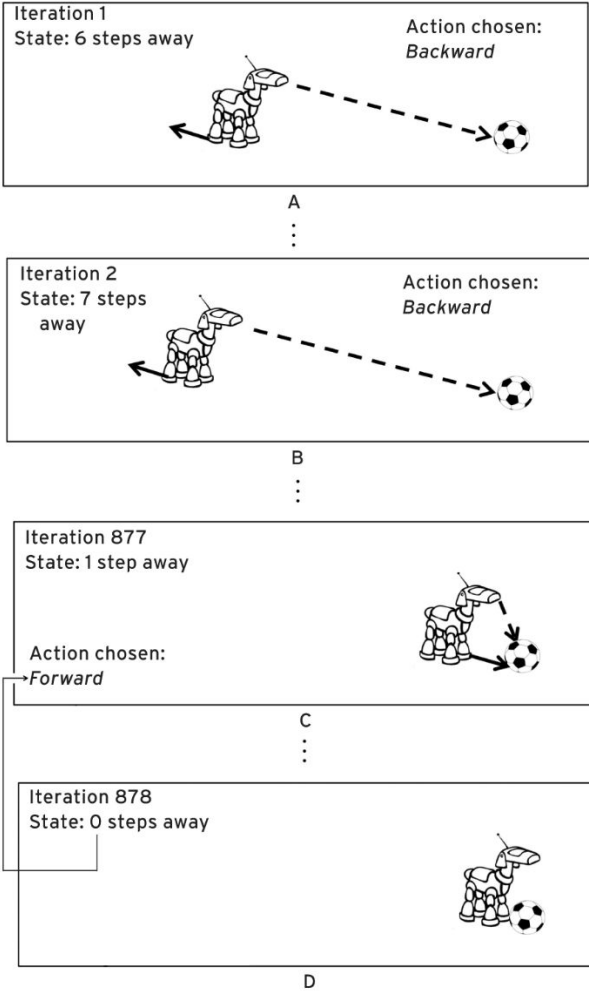


FIGURE 25: The second episode of reinforcement learning

RL Considerations

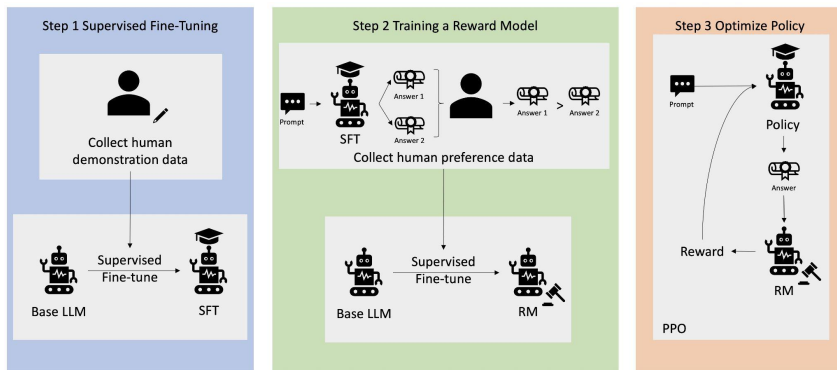
- **exploitation vs exploration balance** - stick with what you know, or risk the unknown and maybe get a better reward?
- how to represent states for infinite problems?
 - remember the combinatorial explosion problem of symbolic AI and the general problem solves
 - solution: use neural nets.

RLHF: The "chat" in ChatGPT

RLHF = Reinforcement Learning from Human Feedback

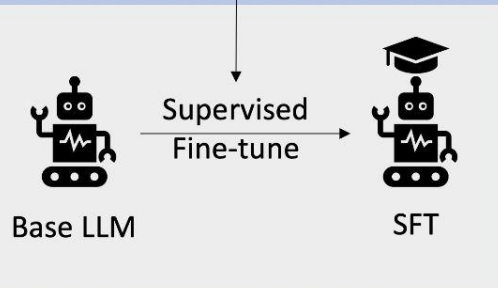
The Three-Stage Process

1. Supervised Fine-Tuning (SFT) - Show the model good examples
2. Reward Model Training - Teach a separate "judge" model what good looks like
3. Reinforcement Learning RL Optimization - Let the model learn to maximize the judge's approval

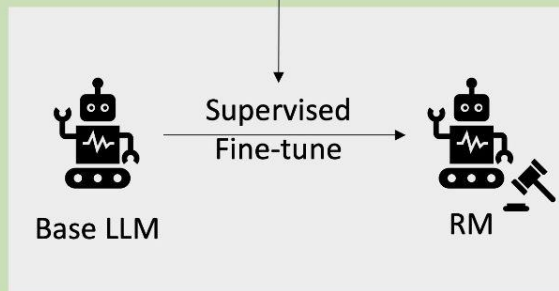
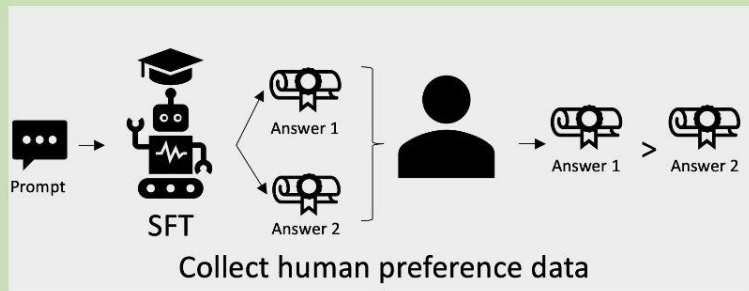


RLHF: The "chat" in ChatGPT

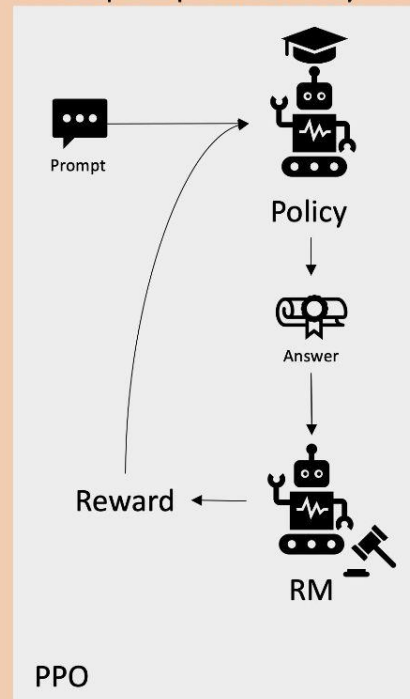
Step 1 Supervised Fine-Tuning



Step 2 Training a Reward Model



Step 3 Optimize Policy



RLHF Stage 1 - Supervised Fine-Tuning

Teaching by example

- Humans write high-quality prompt-response pairs
- "What's the capital of France?" → "The capital of France is Paris."
- Fine-tune the base model on these demonstrations
- Result: A model that imitates helpful behavior

RLHF Stage 2 - The Reward Model

Training a preference judge

- Show humans multiple model outputs for the same prompt
- Ask: "Which response is better?"
- Train a separate neural network to predict human preferences
- This becomes your automated "quality scorer"

RLHF Stage 3 - Reinforcement Learning

Optimizing for human preferences

- Generate responses and score them with the reward model
- Use RL (typically PPO algorithm) to update the LLM
- The model learns: "responses like this get high scores"
- Balance: Don't drift too far from the original model ("KL penalty")

RLHF - Why it Works

- Easier for humans to compare than to generate perfect examples
 - the training data is easier to generate!
- The reward model generalizes beyond the training comparisons
- RL explores the space of possible responses efficiently
 - we don't just tell the model "you're wrong". We tell it how good or bad it's score is.
- Result: Models that are act like chatbots

Scaling LLMs: How big can we go?

GPT-5 optimized performance almost exactly as predicted for this combination of model size / training / compute.

